



UNIVERSIDADE DO OESTE DE SANTA CATARINA – UNOESC
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PRÁTICAS EXTENSIONISTAS IV

Práticas Extensionistas IV:

Desenvolvimento de sistema de gestão de consultas

Desenvolvedor:

Guilherme Victor Colossi

Xanxerê - SC, 27 de Setembro de 2026

1.Introdução e Contextualização

O presente trabalho consiste na continuação do projeto de práticas extensionistas 3, um sistema web voltado à área da saúde pública. O projeto tem como objetivo disponibilizar uma aplicação que facilite o gerenciamento de consultas, pacientes, profissionais, unidades de atendimento e solicitações de medicamentos, reduzindo a dependência de processos exclusivamente manuais e presenciais.

O sistema foi desenvolvido utilizando o framework Django, da linguagem Python, e utiliza banco de dados MySQL para armazenamento das informações. Entre as funcionalidades previstas estão o controle de usuários, cadastro de pacientes, profissionais e unidades, agendamento e gerenciamento de consultas, controle de solicitações de medicamentos e geração de informações relacionadas aos atendimentos.

Como evolução do sistema existente, também é proposta a utilização de conceitos de **Progressive Web Application (PWA)**, permitindo que a aplicação seja acessada por dispositivos como computadores e smartphones por meio de um navegador, com possibilidade de instalação como aplicativo e utilização de recursos próprios de aplicações web modernas.

Para organizar tecnicamente essa evolução, foram elaborados três diagramas: o **Diagrama UML de Pacotes**, o **Diagrama de Arquitetura de Implantação** e o **Diagrama de Arquitetura DevOps**. Cada um apresenta uma visão diferente do sistema, permitindo compreender sua organização interna, o ambiente em que os componentes serão executados e o processo de desenvolvimento, testes e publicação da aplicação.

Além disso, foi escolhida uma infraestrutura de **cloud computing** para a publicação do sistema. A proposta utiliza o GitHub para armazenamento e versionamento do código, o Render para hospedagem da aplicação Django e o Aiven para o banco de dados MySQL. Essa arquitetura busca proporcionar uma solução adequada ao porte do projeto, com menor necessidade de manutenção de infraestrutura física e possibilidade de evolução futura.

2. Link do repositório GitHub

<https://github.com/colossiguilherme05-cpu/Praticas-extensionistas-4>

3. Diagrama UML de Pacotes

O Diagrama UML de Pacotes apresenta a organização lógica da aplicação e a forma como seus principais componentes estão estruturados e

relacionados. A arquitetura foi organizada em diferentes níveis, permitindo representar desde a interface utilizada pelo usuário até a camada responsável pelo armazenamento dos dados.

No primeiro nível está o **PWA (Progressive Web Application)**, que representa a interface de acesso ao sistema. Por meio do PWA, os usuários poderão acessar a aplicação utilizando diferentes dispositivos, como computadores, smartphones e tablets, por meio de um navegador. Essa camada é responsável pela interação entre o usuário e o sistema.

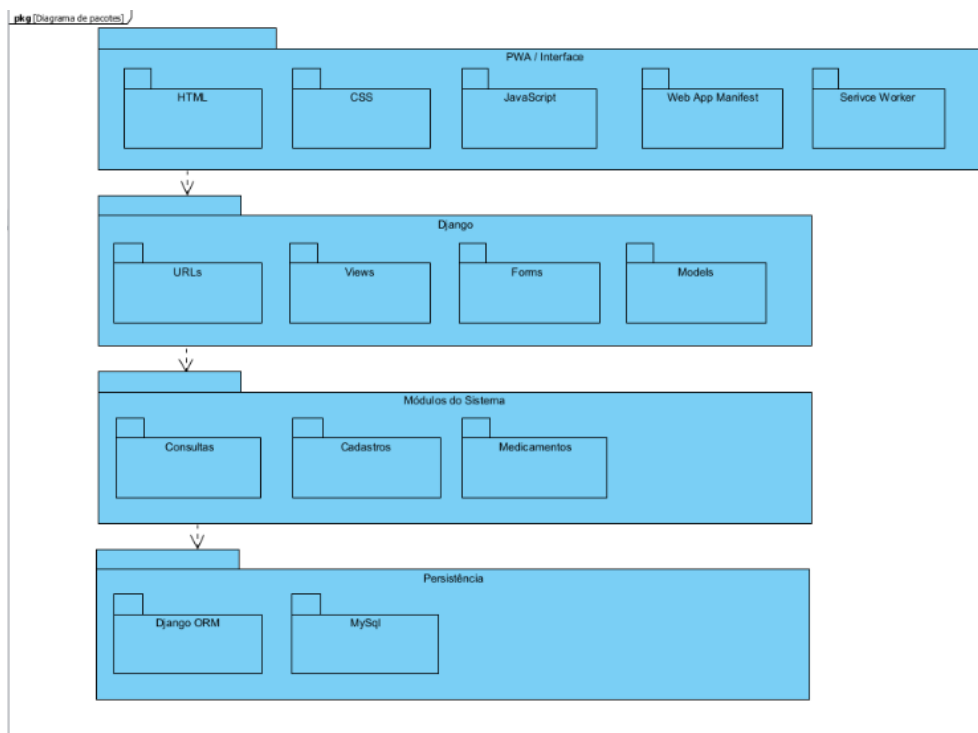
Abaixo do PWA está o **Django**, framework responsável pela estrutura principal da aplicação. O Django concentra o processamento das requisições, as regras de negócio, o gerenciamento das rotas e a comunicação entre a interface e as demais partes do sistema.

Dentro da aplicação Django estão organizados os **módulos do sistema**, responsáveis pelas diferentes funcionalidades oferecidas. Entre eles estão os módulos de Cadastros, Consultas e Medicamentos

Os módulos utilizam a camada de **Persistência de Dados** para armazenar e consultar as informações necessárias para o funcionamento do sistema.

Na persistência, é utilizado o **Django ORM (Object-Relational Mapping)**, recurso do Django que permite realizar a comunicação entre as estruturas utilizadas pela aplicação e o banco de dados relacional. Dessa forma, os módulos do sistema não precisam realizar diretamente comandos SQL para cada operação, pois o ORM realiza a tradução das operações da aplicação para comandos compatíveis com o banco de dados.

Por fim, o **banco de dados MySQL** é responsável pelo armazenamento persistente das informações do sistema, como usuários, pacientes, profissionais, consultas, unidades e solicitações de medicamentos.



4. Diagrama de Arquitetura de Implantação

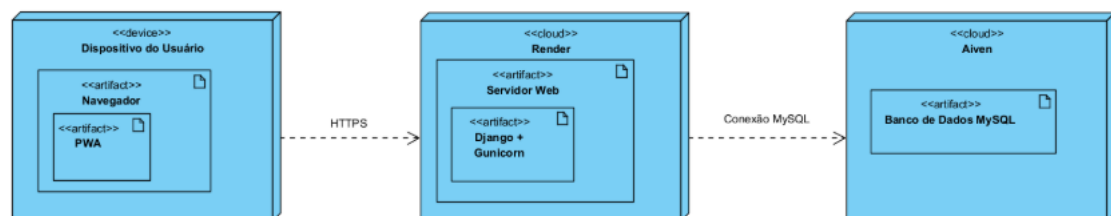
O Diagrama de Arquitetura de Implantação representa **onde os componentes do sistema serão executados** e como ocorre a comunicação entre os dispositivos dos usuários, o servidor da aplicação e o banco de dados.

Na proposta apresentada, o usuário pode acessar o sistema utilizando um computador ou smartphone por meio de um navegador. A aplicação possui características de PWA, permitindo que sua interface seja utilizada em diferentes dispositivos.

A comunicação entre o dispositivo do usuário e o servidor ocorre por meio de **HTTPS**, proporcionando uma comunicação protegida entre o cliente e a aplicação.

A aplicação Django será hospedada em um ambiente de cloud, representado pelo **Render**. Nesse ambiente ficará o servidor responsável por executar a aplicação e disponibilizar o sistema aos usuários.

O banco de dados será hospedado separadamente no **Aiven**, utilizando MySQL. A aplicação hospedada no Render realizará uma conexão segura com o banco para consultar e armazenar as informações do sistema.



5. Diagrama de Arquitetura DevOps

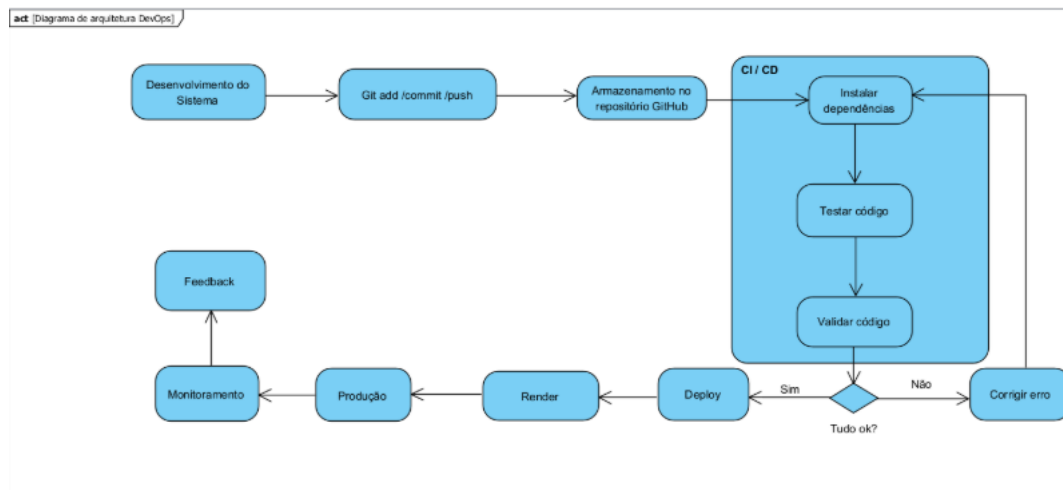
O Diagrama de Arquitetura DevOps representa o processo planejado para desenvolvimento, versionamento, testes e publicação da aplicação.

O fluxo começa no desenvolvimento, quando são realizadas alterações no código da aplicação. Depois, as alterações são registradas utilizando o **Git**, por meio de commits. Posteriormente, o comando push envia essas alterações para o repositório remoto hospedado no **GitHub**.

Após o código chegar ao GitHub, está prevista a utilização de um processo de **CI/CD**. A Integração Contínua (CI) tem como objetivo automatizar verificações e testes da aplicação. Caso os testes sejam aprovados, a etapa

de Entrega/Implantação Contínua (CD) poderá realizar o processo de publicação da nova versão.

O uso de CI/CD é importante porque reduz a quantidade de etapas manuais necessárias para verificar e publicar novas versões do sistema. A proposta também permite identificar problemas antes que uma alteração seja disponibilizada aos usuários.



6. Escolha da Infraestrutura Cloud

Para a implantação e publicação do sistema, foi escolhida uma arquitetura baseada em **computação em nuvem (cloud computing)**. A proposta utiliza dois serviços de nuvem distintos, **Render** e **Aiven**, cada um responsável por uma parte específica da infraestrutura da aplicação.

A arquitetura proposta é composta principalmente por:

- **GitHub**: armazenamento do código-fonte e controle de versões;
- **Render**: hospedagem e execução da aplicação Django;
- **Aiven**: hospedagem do banco de dados MySQL.

A utilização de cloud foi escolhida em vez de uma infraestrutura **self-host** porque o projeto possui características acadêmicas e não necessita, neste momento, da aquisição e manutenção de um servidor físico próprio.

Em uma solução self-host, seria necessário disponibilizar um computador ou servidor permanentemente ligado, além de realizar configurações relacionadas à rede, firewall, acesso externo, segurança, atualizações, armazenamento e disponibilidade. Isso aumentaria a complexidade de administração da infraestrutura.

Na solução em cloud, grande parte dessa infraestrutura é disponibilizada pelos provedores, permitindo que o projeto concentre seus esforços principalmente no desenvolvimento e na manutenção da aplicação.

Outro fator importante para a escolha foi a **disponibilidade de planos gratuitos** adequados ao contexto acadêmico do projeto. A utilização de serviços com opções gratuitas permite desenvolver e apresentar a proposta sem a necessidade de investimentos financeiros iniciais em servidores ou hospedagem.

A escolha de utilizar **dois serviços de cloud diferentes**, em vez de concentrar toda a infraestrutura em um único provedor, ocorre principalmente porque cada serviço possui uma função específica dentro da arquitetura.

O **Render** será utilizado para hospedar e executar a aplicação Django. Dessa forma, ele será responsável pela parte de processamento da aplicação, recebendo as requisições dos usuários e executando o código do sistema.

Já o **Aiven** será utilizado para hospedar o banco de dados **MySQL**, responsável pelo armazenamento persistente das informações da aplicação. Essa separação permite que a aplicação e o banco de dados sejam tratados como componentes independentes.

Outro benefício dessa abordagem é a possibilidade de utilizar serviços especializados para cada camada da aplicação. Caso futuramente seja necessário alterar o provedor da aplicação ou do banco de dados, a arquitetura pode ser modificada de maneira mais independente, sem necessariamente substituir toda a infraestrutura ao mesmo tempo.

Além disso, a utilização de serviços gratuitos é especialmente adequada ao contexto deste projeto acadêmico, pois reduz os custos de desenvolvimento, testes e apresentação.